

Phase 2 Evidence Document

LM Pizzeria - Backend with MySQL & Authentication

Student: Lamita Masry

Course: CSCI 426 - WEH Advanced

Instructor: Prof. Fouad Najem

Term: Spring 2026

Project: LM Pizzeria Restaurant Ordering System

Date: January 2026

Frontend URL: <https://lmpizzeria.netlify.app/>

Backend URL: *To be deployed on Railway/Render*

Phase 2 Requirements Verification

This document provides concrete evidence that the LM Pizzeria project meets all Phase 2 requirements for the university web development course.

Requirement 1: Node.js Backend

Requirement: Build a backend server using Node.js and Express.js.

Evidence:

- Framework: Express.js version 4.18.2
- Runtime: Node.js (ES Module syntax)
- Main Server File: backend/server.js
- Package Configuration: backend/package.json

Server Implementation:

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';

const app = express();
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(Backend server running on port ${PORT});
});
```

Backend Dependencies:

- express: 4.18.2
- cors: 2.8.5
- dotenv: 16.4.5
- mysql2: 3.11.0
- bcrypt: 5.1.1
- jsonwebtoken: 9.0.2

Requirement 2: MySQL Database

Requirement: Use MySQL database for data persistence.

Evidence:

- Database Driver: mysql2 version 3.11.0 (Promise-based)
- Schema File: backend/db/schema.sql
- Connection Module: backend/db/connection.js
- Database Name: lm_pizzeria

Database Connection:

```
import mysql from 'mysql2/promise';

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT || 3306,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```

Requirement 3: Two Related Entities

Requirement: Database must have at least two related entities with foreign key relationships.

Evidence: The database has 3 related tables (exceeds requirement):

Table	Relationships	Purpose
users	One-to-Many with orders	User accounts for authentication
orders	Many-to-One with users, One-to-Many with order_items	Customer orders
order_items	Many-to-One with orders	Individual items within orders

Table 1: Database Schema

Table: users

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_email (email)  
) ENGINE=InnoDB;
```

Table: orders

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  phone VARCHAR(50) NOT NULL,  
  order_type ENUM('dine-in', 'takeaway', 'delivery') NOT NULL,  
  address TEXT,  
  total DECIMAL(10, 2) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  INDEX idx_user_id (user_id)  
) ENGINE=InnoDB;
```

Table: order_items

```
CREATE TABLE order_items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,  
  item_name VARCHAR(255) NOT NULL,  
  item_price DECIMAL(10, 2) NOT NULL,  
  quantity INT NOT NULL DEFAULT 1,  
  FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,  
  INDEX idx_order_id (order_id)  
) ENGINE=InnoDB;
```

Requirement 4: User Authentication

Requirement: Implement user signup and login functionality.

Evidence:

Authentication Files:

- Controller: backend/controllers/authController.js
- Routes: backend/routes/auth.js
- Middleware: backend/middleware/auth.js

Password Hashing (bcrypt):

```
import bcrypt from 'bcrypt';

const saltRounds = 10;
const hashedPassword = await bcrypt.hash(password, saltRounds);

await connection.query(
  'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
  [name, email, hashedPassword]
);
```

JWT Token Generation:

```
import jwt from 'jsonwebtoken';

const token = jwt.sign(
  { userId: user.id, email: user.email },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);

res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
```

Authentication Middleware:

```
export const authRequired = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Authentication required' });
  }

  const token = authHeader.substring(7);
  const decoded = jwt.verify(token, process.env.JWT_SECRET);

  req.user = { id: decoded.userId, email: decoded.email };
  next();
};
```

Method	Endpoint	Purpose	Authentication
POST	/api/auth/signup	Register new user	No
POST	/api/auth/login	Login existing user	No
GET	/api/auth/me	Get current user profile	Yes (JWT)

Table 2: Authentication Endpoints

Requirement 5: CRUD Operations

Requirement: Implement Create, Read, Update, Delete operations on MySQL database.

Evidence:

- Controller: backend/controllers/orderController.js
- Routes: backend/routes/orders.js

Operation	HTTP Method	Endpoint	Function
Create	POST	/api/orders	createOrder()
Read All	GET	/api/orders	getOrders()
Read One	GET	/api/orders/:id	getOrderById()
Update	PUT	/api/orders/:id	updateOrder()
Delete	DELETE	/api/orders/:id	deleteOrder()

Table 3: CRUD Implementation

CREATE Operation:

```
export const createOrder = async (req, res) => {
  const connection = await db.getConnection();
  await connection.beginTransaction();

  try {
    const [orderResult] = await connection.query(
      `INSERT INTO orders (user_id, name, phone, order_type, address, total)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [userId, name, phone, orderType, address, total]
    );

    for (const item of items) {
      await connection.query(
        `INSERT INTO order_items (order_id, item_name, item_price, quantity)
        VALUES (?, ?, ?, ?)`,
        [orderId, item.name, item.price, item.quantity]
      );
    }

    await connection.commit();
    res.status(201).json({ message: 'Order created', order: {...} });
  } catch (error) {
    await connection.rollback();
    throw error;
  }
};
```

READ Operations:

```
export const getOrders = async (req, res) => {
  const [orders] = await db.query(
    SELECT o.*, JSON_ARRAYAGG( JSON_OBJECT('name', oi.item_name, 'price', oi.item_price,
```

```
'quantity', oi.quantity) ) as items FROM orders o LEFT JOIN order_items oi ON o.id =
oi.order_id WHERE o.user_id = ? GROUP BY o.id ORDER BY o.created_at DESC,
[userId]
);

res.json({ orders });

};
```

UPDATE Operation:

```
export const updateOrder = async (req, res) => {
  const { id } = req.params;
  const userId = req.user.id;
  const { name, phone, orderType, address } = req.body;

  const [existing] = await db.query(
    'SELECT id FROM orders WHERE id = ? AND user_id = ?',
    [id, userId]
  );

  if (existing.length === 0) {
    return res.status(404).json({ message: 'Order not found' });
  }

  await db.query(
    `UPDATE orders SET name = ?, phone = ?, order_type = ?,
    address = ?, updated_at = CURRENT_TIMESTAMP WHERE id = ?`,
    [name, phone, orderType, address, id]
  );

  res.json({ message: 'Order updated' });

};
```

DELETE Operation:

```
export const deleteOrder = async (req, res) => {
  const { id } = req.params;
  const userId = req.user.id;

  const [result] = await db.query(
    'DELETE FROM orders WHERE id = ? AND user_id = ?',
    [id, userId]
  );

  if (result.affectedRows === 0) {
    return res.status(404).json({ message: 'Order not found' });
  }

  res.json({ message: 'Order deleted successfully' });

};
```

Requirement 6: Data Validation

Requirement: Implement proper input validation and sanitization.

Evidence:

- Validation Function: validateOrderData() in orderController.js
- Location: backend/controllers/orderController.js

Validation Rules:

```
const validateOrderData = (data) => {  
  const { name, phone, orderType, items, total } = data;  
  const errors = [];  
  
  if (!name || typeof name !== 'string' || name.trim().length === 0) {  
    errors.push('Name is required');  
  }  
  
  if (!phone || typeof phone !== 'string' || phone.trim().length === 0) {  
    errors.push('Phone number is required');  
  }  
  
  if (!orderType || ![ 'dine-in', 'takeaway', 'delivery' ].includes(orderType)) {  
    errors.push('Order type must be: dine-in, takeaway, or delivery');  
  }  
  
  if (orderType === 'delivery' && (!data.address || data.address.trim().length === 0)) {  
    errors.push('Delivery address is required for delivery orders');  
  }  
  
  if (!items || !Array.isArray(items) || items.length === 0) {  
    errors.push('Order must contain at least one item');  
  }  
  
  if (typeof total !== 'number' || total <= 0) {  
    errors.push('Valid total amount is required');  
  }  
  
  return errors;  
};
```

SQL Injection Prevention:

- Uses parameterized queries throughout
 - Example: db.query('SELECT * FROM orders WHERE id = ?', [id])
 - Never concatenates user input into SQL strings
-

Requirement 7: Error Handling

Requirement: Implement comprehensive error handling with appropriate HTTP status codes.

Evidence:

- Error Middleware: backend/middleware/errorHandler.js
- Implementation: Registered in server.js as final middleware

Error Handler Implementation:

```
export const errorHandler = (err, req, res, next) => {
  console.error('Error:', err);

  const status = err.status || err.statusCode || 500;
  const message = err.message || 'Internal server error';

  const response = { message };

  if (process.env.NODE_ENV === 'development') {
    response.stack = err.stack;
  }

  if (err.details) {
    response.details = err.details;
  }

  res.status(status).json(response);
};
```

Code	Situation	Example
200	Success	Order retrieved successfully
201	Created	New order/user created
400	Bad Request	Validation errors
401	Unauthorized	Invalid/missing JWT token
403	Forbidden	Token expired
404	Not Found	Order/user not found
500	Server Error	Database connection failed

Table 4: HTTP Status Codes

Requirement 8: Updated Documentation

Requirement: Update README with backend setup instructions, API documentation, and screenshots.

Evidence:

- File: [README.md](#)

- Sections Included:
 - Backend setup instructions (MySQL + npm install + .env config)
 - API endpoint documentation with examples
 - Database schema documentation
 - Deployment guides (Railway/Render)
 - Environment variables documentation
 - Security features documentation
 - Testing instructions

README Contents:

- Total Lines: ~450 lines
- API Endpoints: 8 documented with cURL examples
- Setup Steps: 6-step process
- Database Schema: Tables and relationships documented
- Deployment: Separate sections for frontend (Netlify) and backend (Railway/Render)

API Endpoints Summary

Authentication Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/auth/signup	No	Register new user
POST	/api/auth/login	No	Login and get JWT token
GET	/api/auth/me	Yes	Get current user profile

Table 5: Authentication Endpoints

Order Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/orders	Yes	Create new order
GET	/api/orders	Yes	Get all orders for user
GET	/api/orders/:id	Yes	Get specific order
PUT	/api/orders/:id	Yes	Update order details
DELETE	/api/orders/:id	Yes	Delete order

Table 6: Order Endpoints

Technology Stack (Phase 2)

Backend Technologies

Technology	Version	Purpose
Node.js	Latest	JavaScript runtime
Express.js	4.18.2	Web framework
MySQL	8.0+	Relational database
mysql2	3.11.0	MySQL driver (Promise-based)
bcrypt	5.1.1	Password hashing
jsonwebtoken	9.0.2	JWT authentication
dotenv	16.4.5	Environment variables
cors	2.8.5	Cross-origin resource sharing
nodemon	3.1.0	Development auto-reload

Table 7: Backend Technologies

Security Features

- **Password Security:** bcrypt hashing with 10 salt rounds, never stores plain-text passwords, password verification on login
- **JWT Authentication:** 7-day token expiration, Bearer token in Authorization header, token verification on protected routes
- **SQL Injection Prevention:** Parameterized queries throughout, no string concatenation in SQL
- **Authorization:** Users can only access their own orders, ownership verification before update/delete operations
- **CORS Configuration:** Restricted to frontend domain, credentials support enabled
- **Error Handling:** No sensitive data in error messages, stack traces only in development mode
- **Environment Variables:** Secrets stored in .env (not committed), .env.example templates provided
- **Input Validation:** Type checking and sanitization, business rule validation

Database Setup Instructions

1. Create Database

```
mysql -u root -p
CREATE DATABASE lm_pizzeria;
```

2. Run Schema

```
mysql -u root -p lm_pizzeria < backend/db/schema.sql
```

3. Verify Tables

```
USE lm_pizzeria;  
SHOW TABLES;  
DESCRIBE users;  
DESCRIBE orders;  
DESCRIBE order_items;
```

Local Testing Workflow

1. Start Backend

```
cd backend  
npm install  
npm run dev
```

2. Start Frontend

```
npm install  
npm run dev
```

3. Test Authentication

1. Visit <http://localhost:5173/signup>
2. Create new account
3. Login with credentials
4. Verify token in localStorage

4. Test CRUD Operations

1. Add items to cart
 2. Checkout (creates order)
 3. View orders page (reads orders)
 4. Delete an order
-

Phase 2 Checklist Summary

- ✅ Node.js Backend - Express.js 4.18.2 server
- ✅ MySQL Database - mysql2 driver with connection pool
- ✅ Two Related Entities - users, orders, order_items with foreign keys
- ✅ User Authentication - bcrypt + JWT implementation
- ✅ CRUD Operations - All 5 operations on orders
- ✅ Data Validation - Comprehensive input validation

- ✅ Error Handling - Middleware with proper HTTP codes
 - ✅ Updated Documentation - Complete README with API docs
 - ✅ Frontend Integration - Login, Signup, Orders pages
 - ✅ Security - bcrypt, JWT, parameterized queries, CORS
-

Conclusion

This evidence document demonstrates that the LM Pizzeria project fully satisfies all Phase 2 requirements:

1. Built with Node.js and Express.js backend
2. MySQL database integration with proper schema
3. Three related entities with foreign key relationships
4. Complete authentication system (signup/login with bcrypt + JWT)
5. Full CRUD operations on orders
6. Comprehensive data validation
7. Professional error handling with proper HTTP status codes
8. Updated documentation with API examples and setup instructions
9. Frontend integration with authentication state
10. Security best practices (password hashing, JWT, SQL injection prevention)

All files referenced in this document exist in the repository and can be verified by inspecting the codebase.

Document Version: 1.0

Last Updated: January 2026

Prepared By: Lamita Masry